

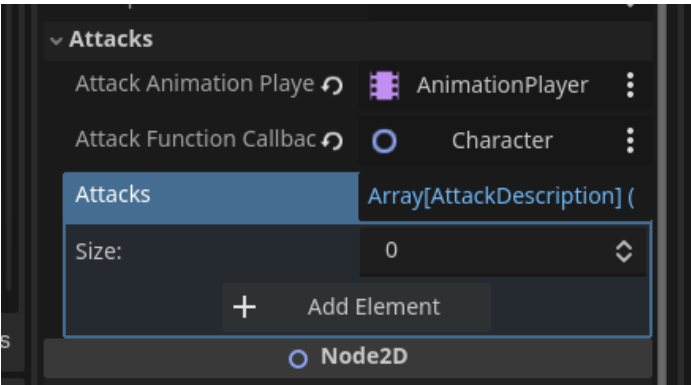
This is the part 2 of “How to create a new character” tutorial.

Contents

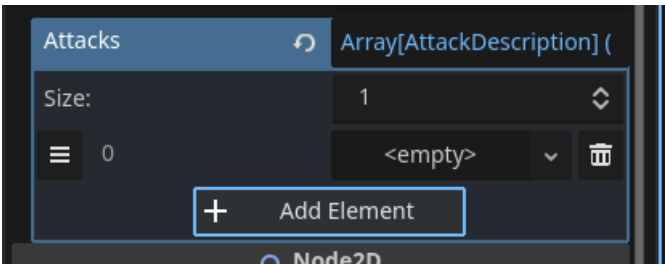
Adding new attacks.....	1
Making a boss from the character	13
Making the character fly (optional).....	19
Separating the top half of the character sprites (optional).....	21

Adding new attacks

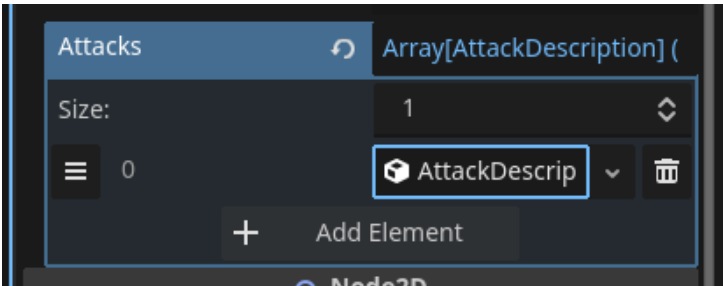
To add a new attack, open the character’s skin scene, open the main node’s properties and you’ll see it has a list of attacks:



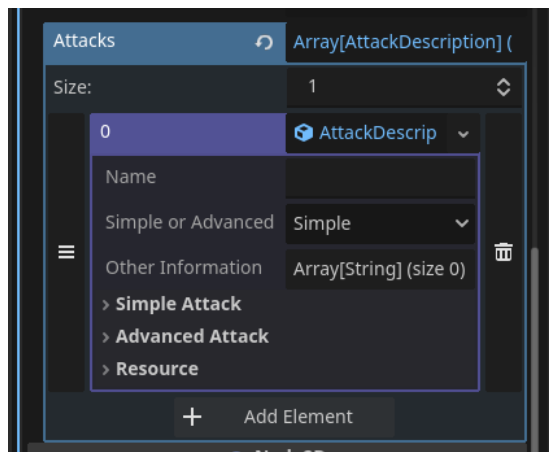
Let’s add a new one. Press “Add Element”:



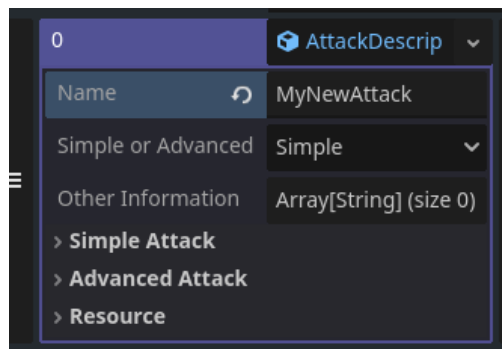
Now press on “<empty>” and then on “New AttackDescription”:



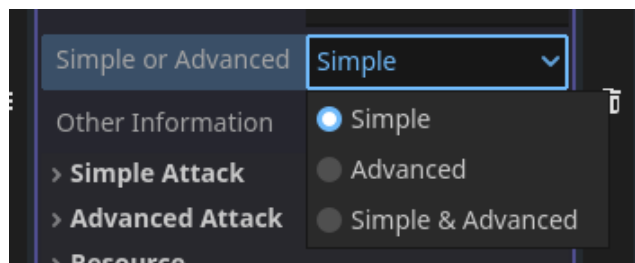
Open the attack description by clicking on it:



Let's add a name for our attack:



There are 3 types of attacks in Godotzilla: simple, advanced and simple & advanced.



First of all, what is a “simple” attack? Well, if you select this type and open the “Simple Attack” group, you’ll see a bunch of attack properties:

The screenshot shows the 'AttackEditor' window for an attack named 'MyNewAttack'. The 'Simple or Advanced' dropdown is set to 'Simple'. Under the 'Simple Attack' section, the 'Type' is 'One Time', 'Damage Amount' is 0.0, 'Hitbox Name' is empty, and 'Sfx' is '<empty>'. The 'Animation And Timing' section shows 'Hurt Time' as -1.0, 'Start Time Offset' as 0.0, 'Animation Name' and 'Animation Name 2' are empty, 'Time Length' is -1.0, and both 'Reset Animation B' and 'Reset Animation A' are checked and set to 'On'. The 'Advanced Attack' and 'Resource' sections are collapsed.

So a “simple” attack is essentially an attack that uses a premade attack template with customizable parameters, while an advanced attack is the one made fully by coding (like Godzilla’s heat beam or Mothra’s attacks), and “simple & advanced” attack is a mix of those: it’s a simple attack but it can also run custom attack code.

So, first let me explain the simple attacks.

There are also 3 types of simple attacks: “One Time”, “Continuous” and “Lasts Forever”.

This close-up shows the 'Simple Attack' section with the 'Type' dropdown menu open. The 'One Time' option is selected and highlighted with a blue border. The other options, 'Continuous' and 'Lasts Forever', are visible but not selected. The 'Damage Amount' is 0.0, 'Hitbox Name' is empty, and 'Sfx' is '<empty>'. The 'Reset Animation B' and 'Reset Animation A' checkboxes are checked.

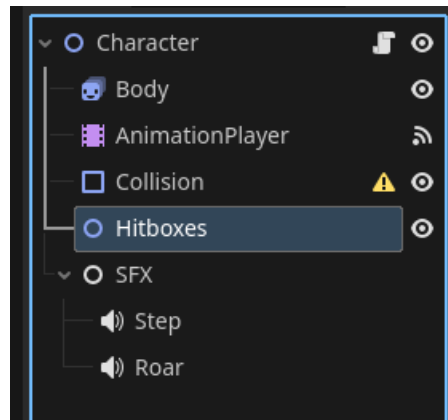
“One Time” attack is an attack that hits other entities only once and after that the attack stops or waits until the attack animation is finished. Most of Godzilla attacks are one time: punch, kick and tail whip.

“Continuous” attack is basically a “One Time” attack but it always checks for a new body to attack until the attack is finished.

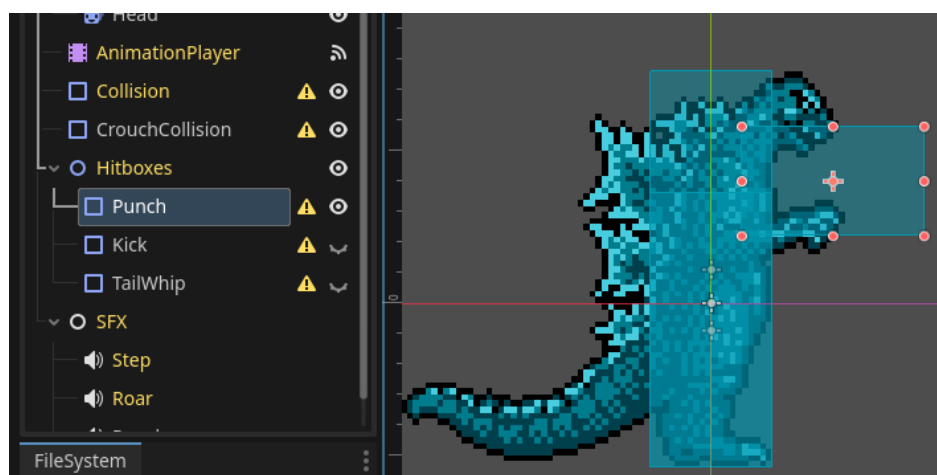
“Lasts Forever” attack is basically a “Continuous” attack but it doesn’t automatically finish after the attack animation is done (if it’s present at all).

Now onto other properties. “Damage Amount” does what you think it does: it’s the number of HP that will be reduced from other body’s health component.

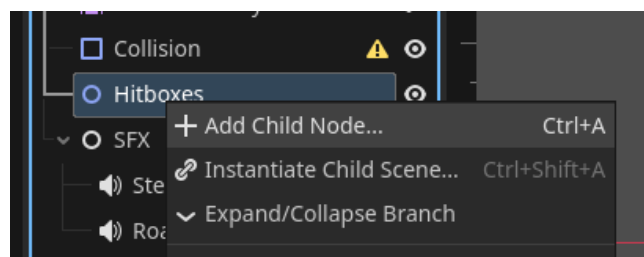
“Hitbox Name” is the name of the attack’s hitbox. You can add new hitboxes for attacks by adding CollisionShape2D’s to the “Hitboxes” node of the character:



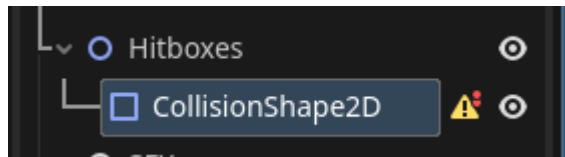
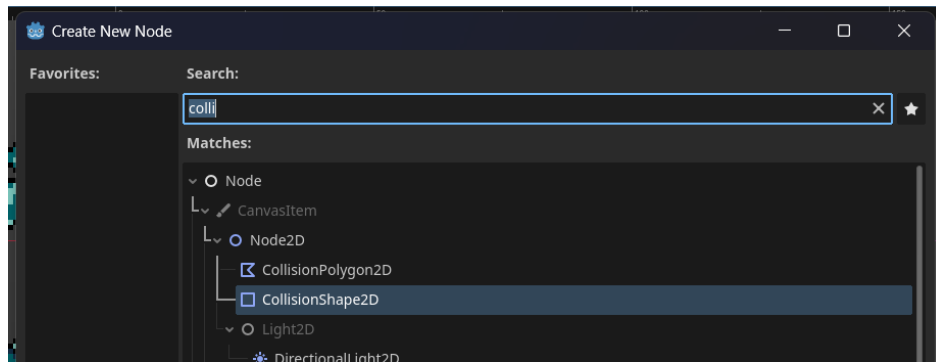
You can check how it’s done for Godzilla:



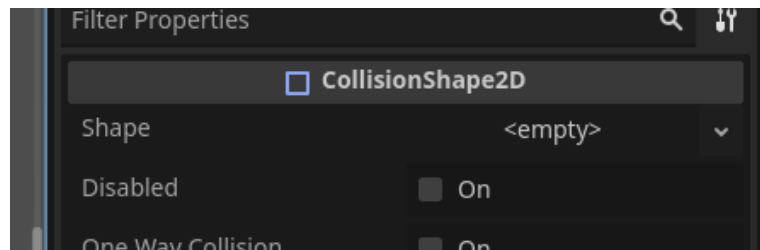
So let’s add our own hitbox. Right click on the hitboxes node and add a new child node:



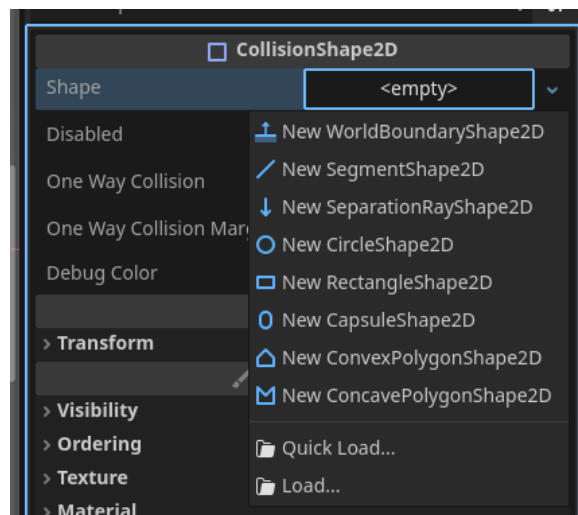
It should be a CollisionShape2D node:



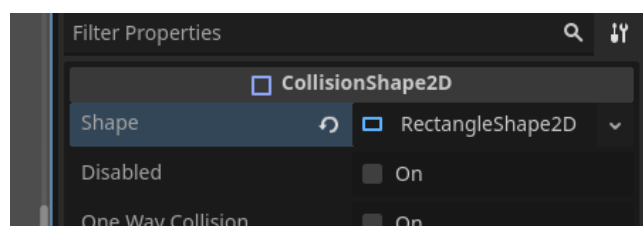
Godot is complaining because we haven't added the actual shape data yet. Go to the new node's properties:



Here's the shape property. Click on "<empty>":



There are many collision shapes but I usually use RectangleShape2D most of the time.



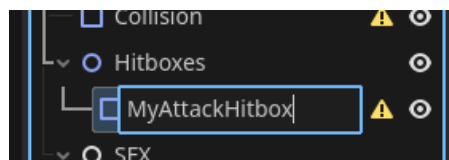
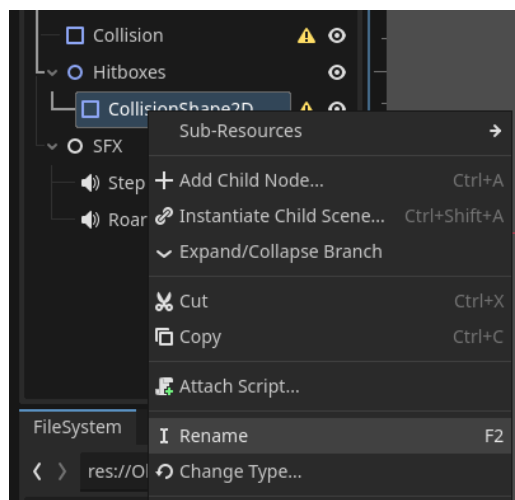
A new rectangle appeared in the scene:



That's the attack hitbox. You can move and resize it however you like:

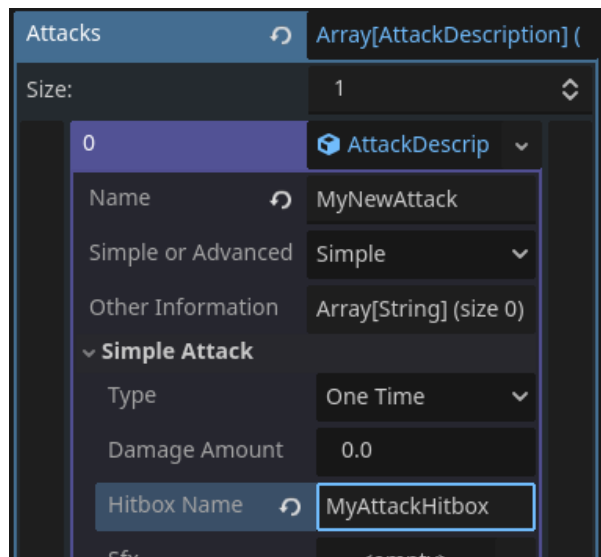


Now let's give it a proper name. Right click on the hitbox shape node and then on "Rename":



The name can be anything you want, but I usually name my hitboxes after the attacks they will be used for.

Now let's go back to the attack's properties in the character's node and specify the hitbox's node name:



Nice. Now we can go to other properties.

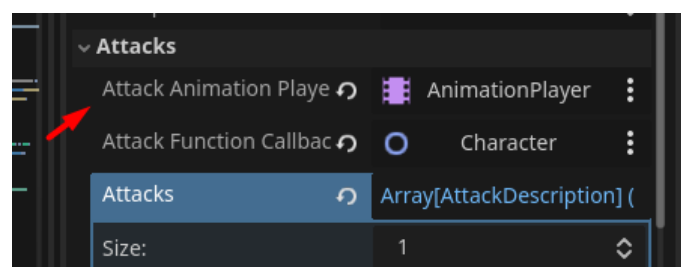
“Sfx” is also pretty self-explanatory, it's the sound effect that will be played when the attack starts. You can drag and drop your sound effect here.

“Hurt Time” is the number of seconds the player will be in a hurt state. You can make the player get hurt for longer, ignore the attack altogether (by setting it to 0) or use the default time by setting it to -1.0.

“Start Time Offset” is when the actual attacking should happen after the attack component started the attack. For example, we can make the component first start the animation and after some time start attacking enemies. It's used by Godzilla's Tail Whip attack.

“Animation Name” is the animation that should start when the attack starts. It can be omitted if you either specify the attack's time length (described below) or change the attack's type to “Lasts Forever”.

How does the attacking component know what animation player it should use for animations? It's specified in the character properties:



“Animation Name 2” is used for animation variation. It’s used by Godzilla’s kicking and punching attacks to swap between attacking with his left arm/leg to his right arm/leg.

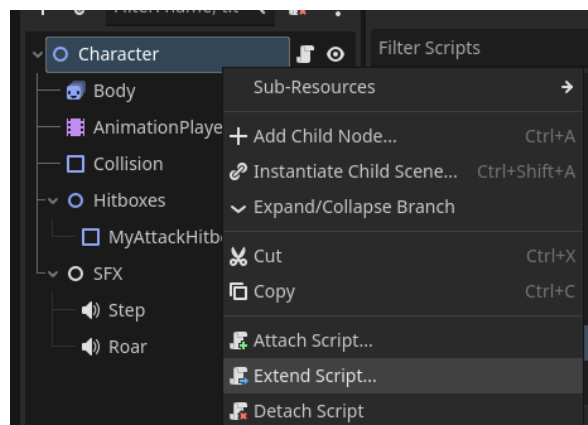
“Time Length” is the length of the attack in seconds. If you don’t really care about making the attack longer than the specified attack animation, set it to -1.0.

“Reset Animation Before” and “Reset Animation After” change the animation player’s animation to “RESET” before or after the attack respectively. If you’re not sure what this is for, you can ignore these properties.

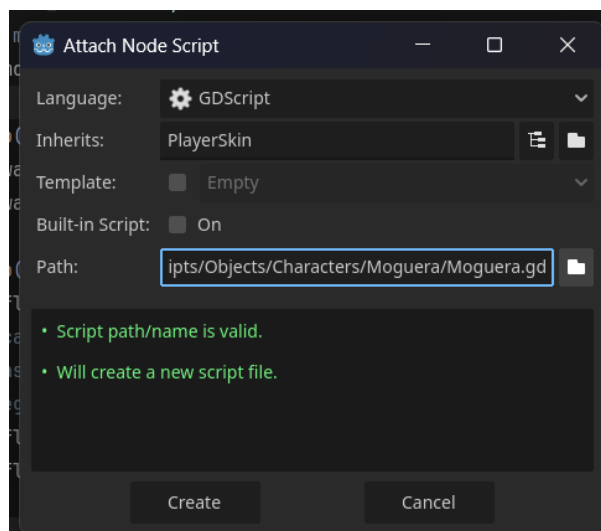
I don’t have an example animation for this attack, so I will skip the animation name properties, set time length to 1 and add some sound effect as an example.

If you play the game, you will notice that you can’t use this new attack yet, that’s because you need to call this attack from code. Let’s add some code for that then!

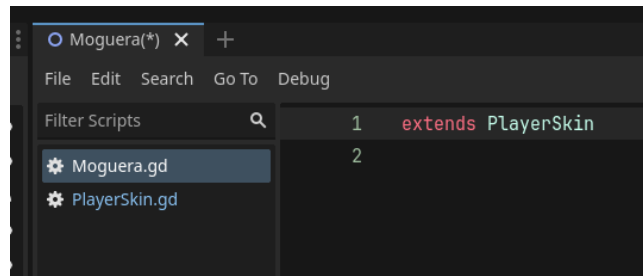
Moguera uses the default character script at the moment, so I will need to create a new one. Right click on the character’s node and then on “Extend Script”:



Place the script somewhere in your project, preferably in “Scripts/Objects/Characters/<your character name here>” folder (if it doesn’t exist you can create it).



Now press “Create”.



You can see how Godzilla starts his attack by opening “Scripts/Objects/Characters/Godzilla/Main.gd” file and looking at walk_process() function:

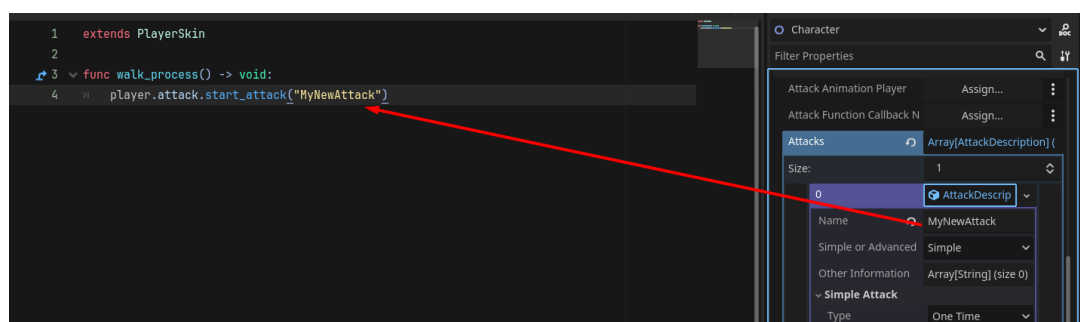
```
24 func walk_process() -> void:
25     common_ground_attacks()
26     if player.animation_player.current_animation == "Crouch" \
27     and player.inputs_pressed[PlayerCharacter.Inputs.B]:
28         player.attack.start_attack("TailWhip")
29     if player.inputs_pressed[PlayerCharacter.Inputs.START] \
30     and player.power.value >= 6 * 8:
31         player.attack.start_attack("HeatBeam")
```

We can borrow some of this code to use in our character’s script, specifically the start_attack() call and the function name.

Go back to your character’s script file. Add a new function called “walk_process” (you can copy that line from Godzilla’s code too), and paste the start_attack line inside:

```
1 extends PlayerSkin
2
3 func walk_process() -> void:
4     player.attack.start_attack("TailWhip")
```

Change the attack name:



Now let’s add a condition, like pressing the A button.

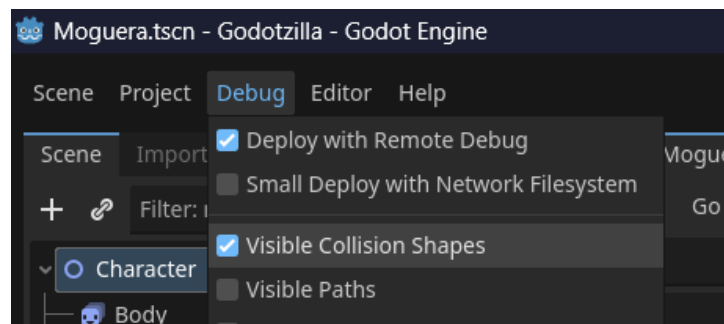
```

1  extends PlayerSkin
2
3  func walk_process() -> void:
4      if Input.is_action_just_pressed("A"):
5          player.attack.start_attack("MyNewAttack")
6

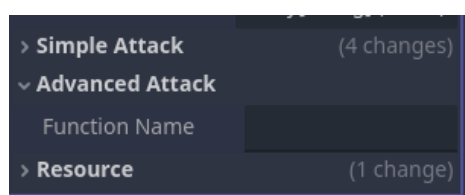
```

Now when I press the A action I can hear the sound effect I specified and I can destroy enemies. If you can't hear anything or destroy enemies, make sure you specified the Sfx and Damage Amount properties.

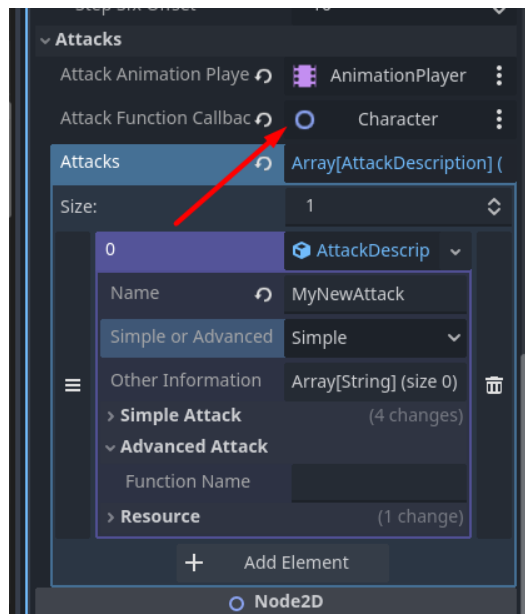
If you want to see your attack hitbox in action, you can enable visible collision shapes like this:



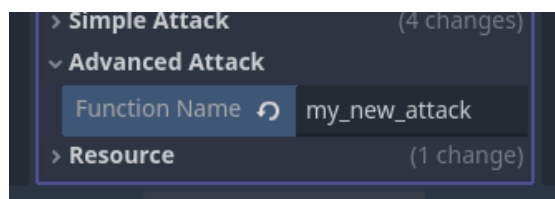
Now I will try to explain what “Advanced” attack means. When you open the “Advanced Attack” group you can see the function name text box:



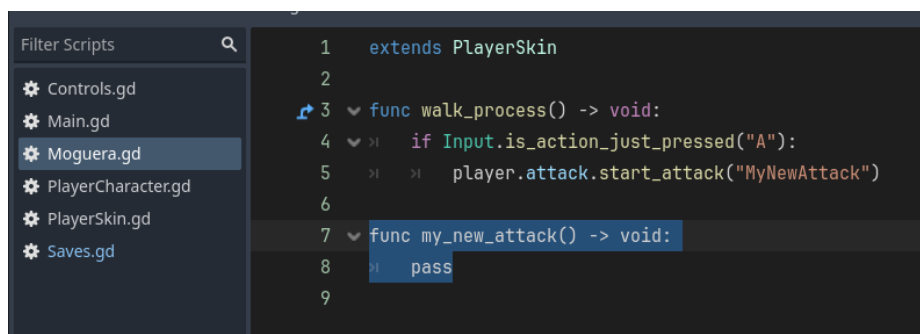
But first let's take a look at this property:



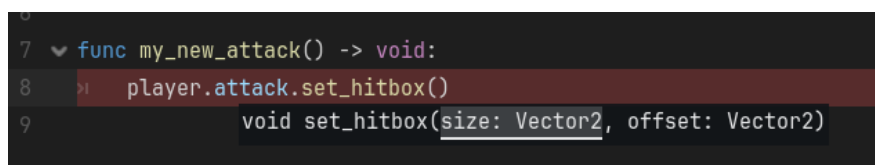
Here we can specify a node. Nodes' scripts have functions (or "methods"). We can call those functions here instead of making a simple attack. You just need to specify the function name in the text box:



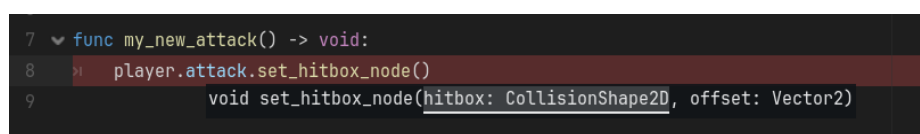
Now let's create this function inside of the node we specified earlier:



Now you can do whatever you want with animations of the player or do anything with the attack component, like setting its collision box:



You can also use a CollisionShape2D instead:



It can be helpful in case you already have a collision box node and you would like to use it for an attack as well.

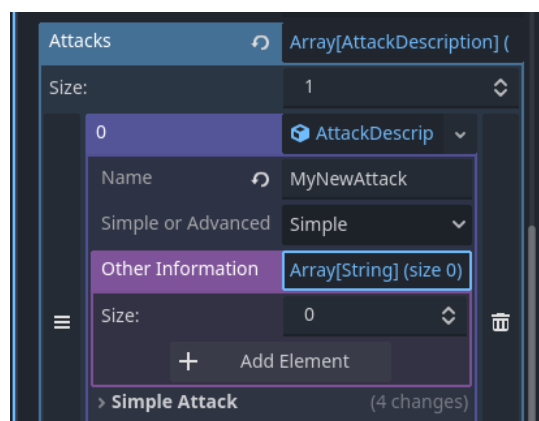
You can also use one of the hitboxes inside of “Hitboxes” node by their names! Not sure why you would use that inside of code, but you can do that too.

```
7 func my_new_attack() -> void:
8     player.attack.set_hitbox_template()
9     void set_hitbox_template(template_name: String)
```

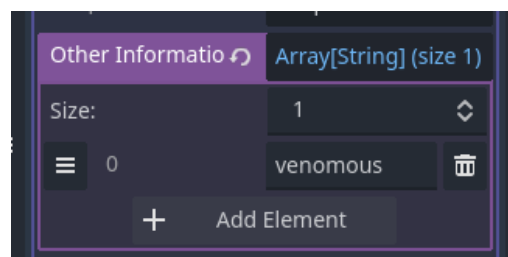
To actually attack the bodies inside of the hitbox using the attack component you can call the “attack_bodies” method:

```
6
7 func my_new_attack() -> void:
8     player.attack.attack_bodies()
9
```

You probably also noticed the “Other Information” property inside of an attack:



If you do something special with the attack, like making it a venomous attack or a freezing attack, you name it, you can put a string in this array to specify the special properties of the attack:



You can then check for these strings inside of other objects that are being attacked:

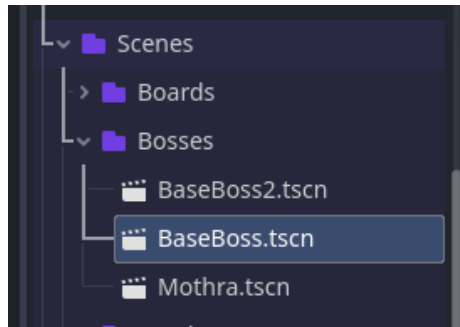
```
func _on_health_component_damaged(_amount: float, attack: AttackDescription) -> void:
    if "venomous" in attack.other_information:
        print("Oh I got bit by a snake! NooOoo!")
```

Making a boss from the character

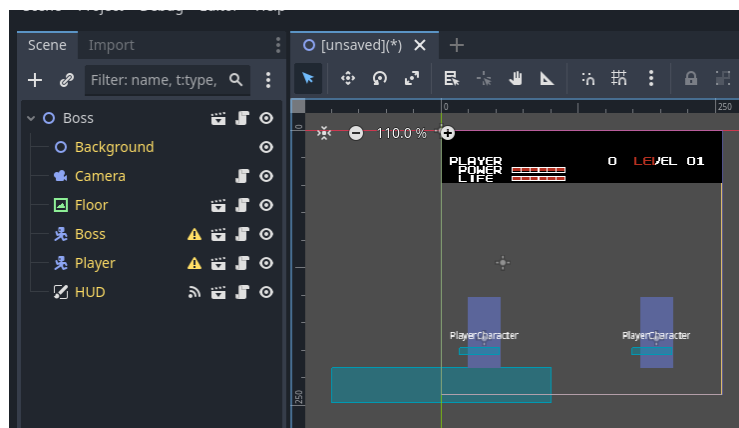
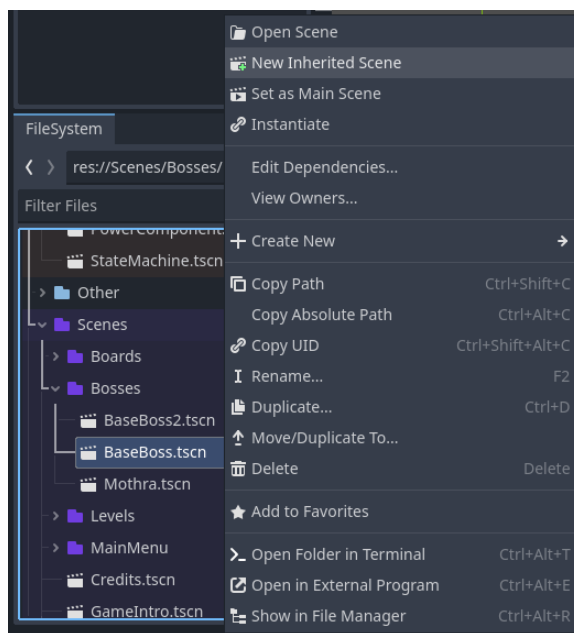
In this section I explain how to make a boss from an existing PlayerCharacter. If you would like to make a boss that is not playable and more simple to setup, you can check out the “How to make a new boss” tutorial.

Yes, the PlayerCharacter can be both players and bosses. Yes, that’s how Mothra boss was done :D

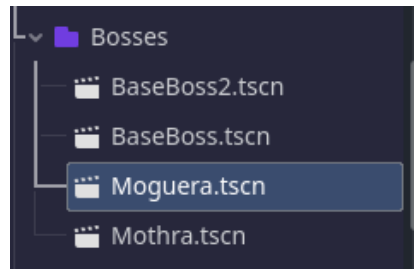
Find this scene file: “Scenes/Bosses/BaseBoss.tscn”



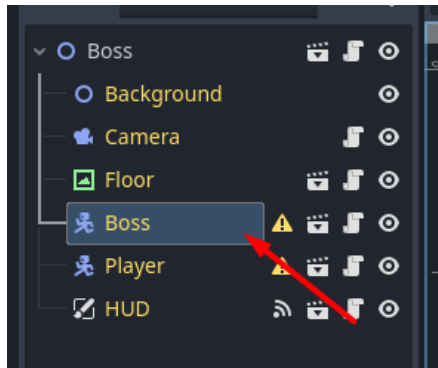
Let’s make a new scene that is based on this scene.



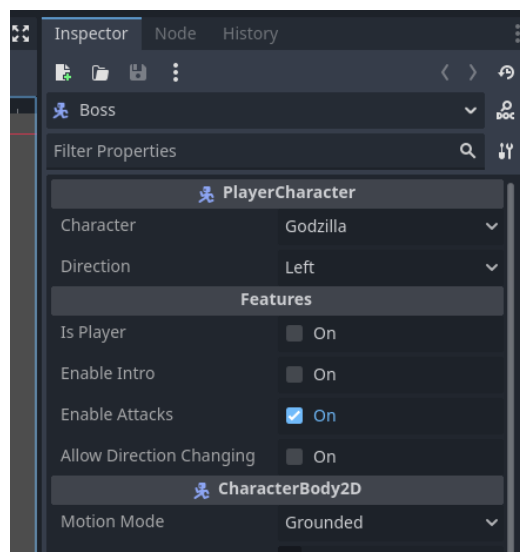
Save this scene to a file.



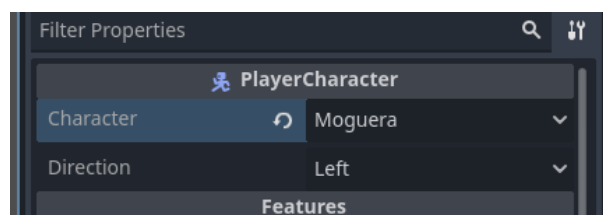
Now let's change the boss here to Moguera. Click on this node:



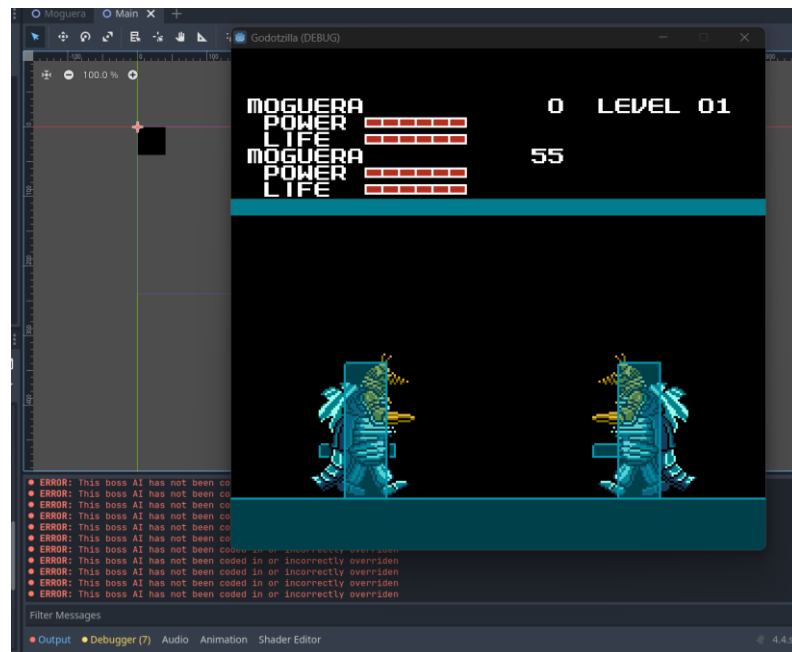
If you look at its properties, it's an actual PlayerCharacter.



You know what to do.

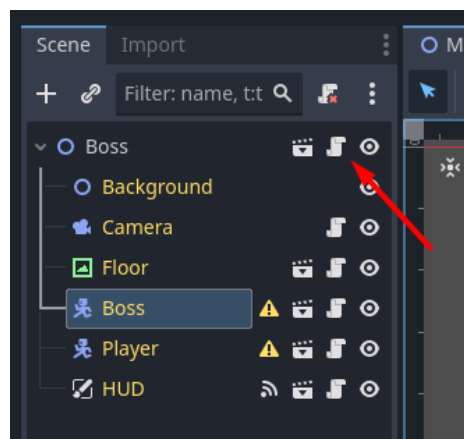


Now let's try it. Change the game's initial scene, change the level's default character back to Godzilla, and...



You can now see the character as a boss! But the output is flooding with errors, because we haven't coded an AI yet.

If you go back to the Moguera boss scene you may notice there's a script attached to it:



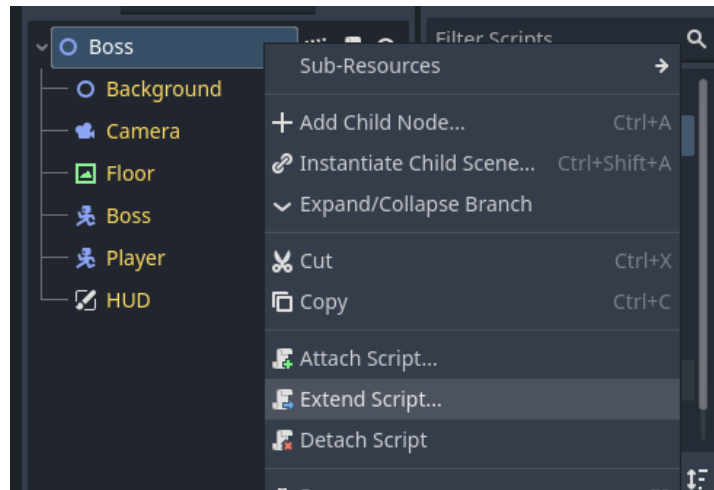
If you open it and scroll down, you can see the boss AI template functions. But don't change the code in those yet!

```

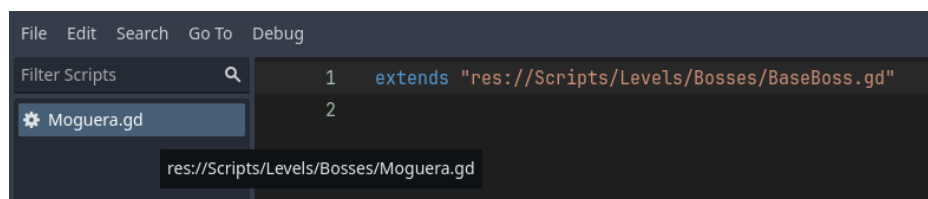
52 >|
53 ▾ func boss_ai_start() -> void:
54   >| pass
55
56 ▾ func boss_ai() -> void:
57   >| printerr("This boss AI has not been coded in or incorrectly overridden")
58
59 ▾ func boss_ai_stop() -> void:
60   >| pass
61

```

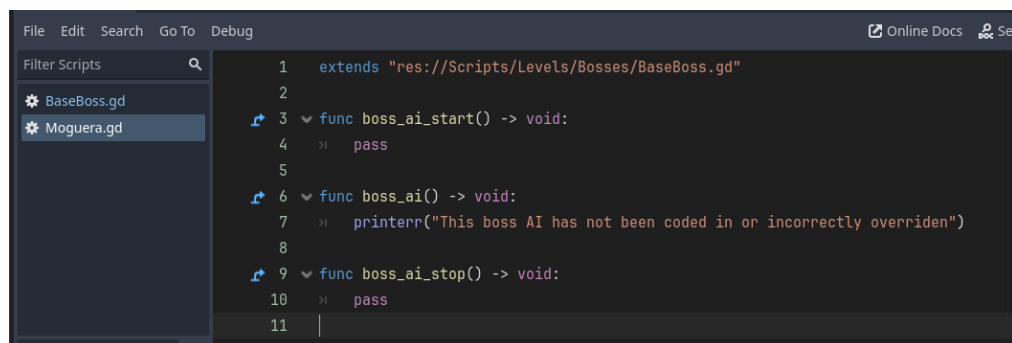
First, let's make a new script for the Moguera boss. Right click on "Boss" node and then "Extend Script..."



Save the new script somewhere, preferably in “Scripts/Levels/Bosses” folder.



Now let's overwrite the boss AI functions, open the parent script, you can do that by clicking on that text after “extends” while holding Ctrl key. Copy the AI function templates and paste them inside your new script:



So now I can explain what each function does.

“boss_ai_start” gets called right when the main player’s intro ends and the actual boss fight starts, here you can make the boss start your AI code.

“boss_ai” is the actual AI code. Preferably it should be disabled before the actual boss fight starts or after the character dies, you can do that by having a “waiting” state for the boss and checking for the character’s state:

```

1 extends "res://Scripts/Levels/Bosses/BaseBoss.gd"
2
3 enum {
4     NONE,
5     IDLE,
6 }
7
8 var state := NONE
9
10 func boss_ai_start() -> void:
11     state = IDLE
12
13 func boss_ai() -> void:
14     if state == NONE or boss.state.current == boss.State.DEAD:
15         return
16     # Boss AI code here
17
18 func boss_ai_stop() -> void:
19     pass
20

```

You can check out how Mothra boss is done so you can have a better understanding of how bosses work. Let's see:

```

match state:
    IDLE:
        boss.inputs[boss.Inputs.XINPUT] = 0
        boss.inputs[boss.Inputs.YINPUT] = 0
        if time <= 0:
            state = MOVING
            time = 20
        boss.inputs[boss.Inputs.XINPUT] = randi_range(-1, 1)
        boss.inputs[boss.Inputs.YINPUT] = randi_range(-1, 1)

```

So, the boss AI is done by simulating input presses for the boss as if someone is actually controlling a playable character, and in this case “someone” is the code we write for the boss AI.

But why are we specifying a number for X and Y input instead of, you know, simulating pressing Up or Left? That's because internally directional presses are done with numbers, so pressing left is making `player.inputs[player.Inputs.XINPUT]` equal to -1, pressing down is setting `player.inputs[player.Inputs.YINPUT]` to 1 and etc.

You can also notice that under certain conditions the boss simulates the Start button:

```

if attack_time > 150 and boss.power.value > 3 * 8:
    attack_time = 0
    boss.simulate_input_press(PlayerCharacter.Inputs.START)

```

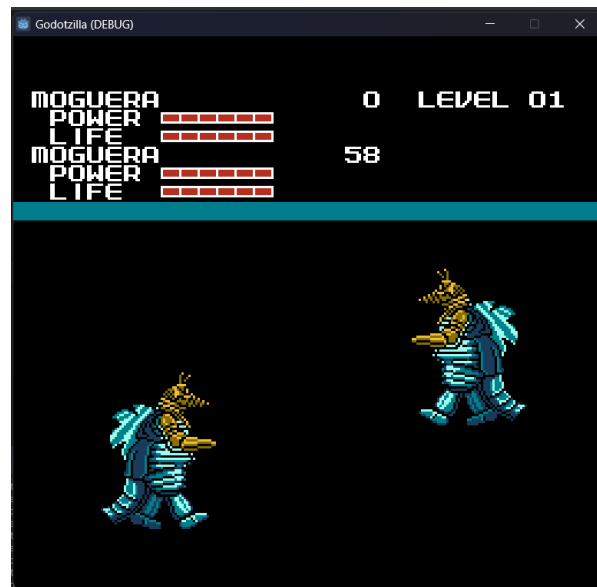
That's how we can make the attacks for our boss. We also just simulate input presses.

Now you can code your own boss AI!

Check this out:

```
12
13 func boss_ai() -> void:
14     if state == NONE or boss.state.current == boss.State.DEAD:
15         return
16     boss.inputs[boss.Inputs.YINPUT] = -1
17
```

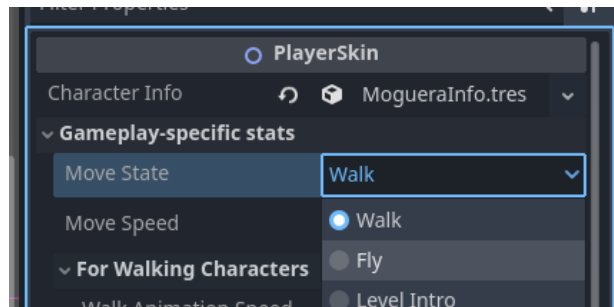
Now Moguera will always be jumping:



Making the character fly (optional)

If you want to create a flying character, like Mothra or Rodan, we should do the following:

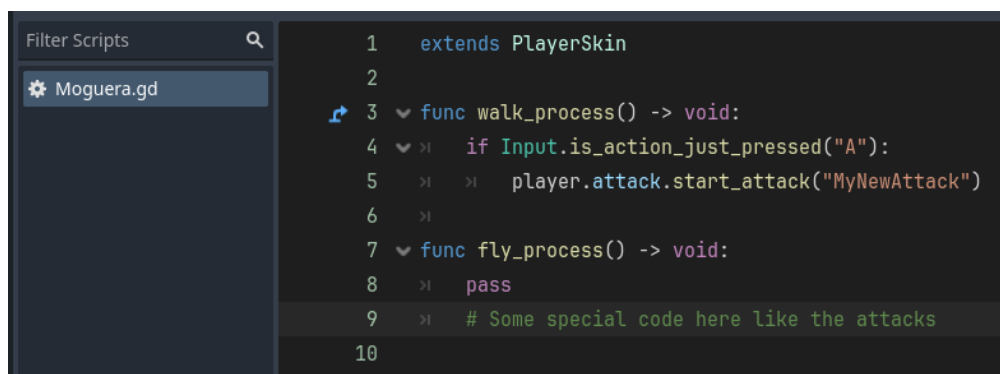
Open the character's skin and open its properties. In the "Gameplay-specific stats" you can see the "Move State" property. Change it to "Fly":



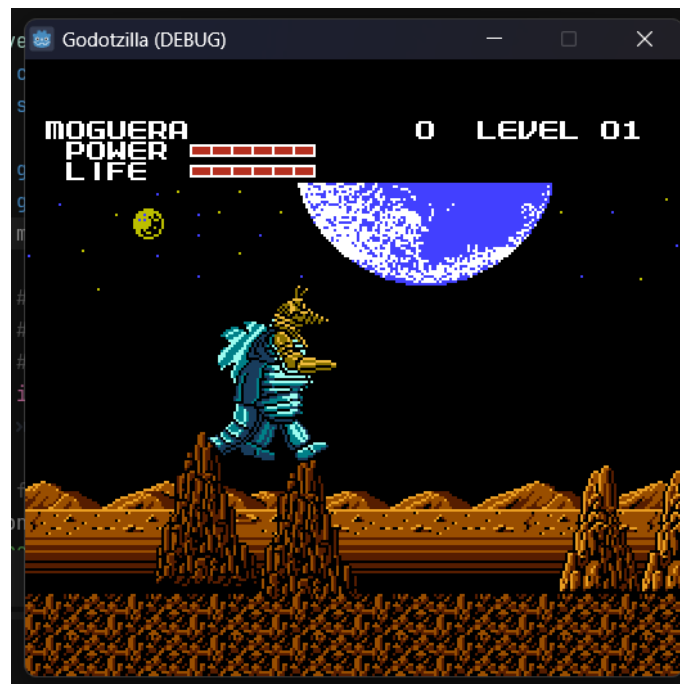
If you try to play a level now the game will crash:



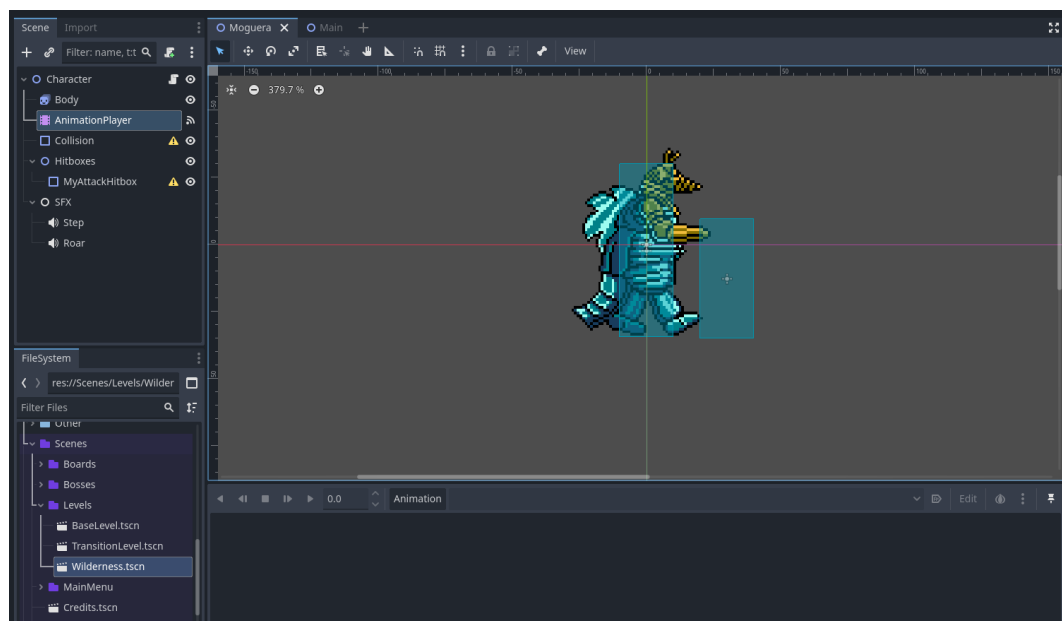
Pretty self-explanatory, we had a process function for walking but this time we need a flying one. Open the character skin's script and create a new "fly_process" function:



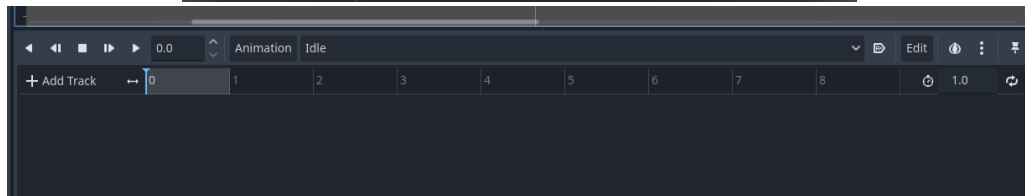
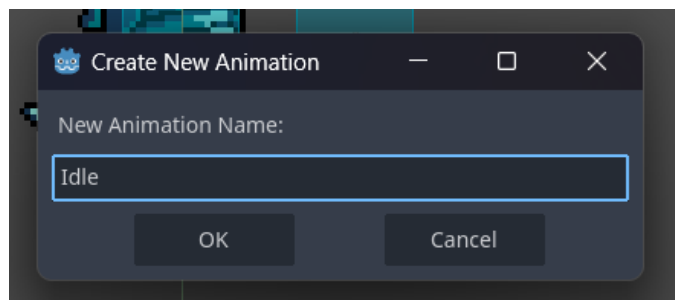
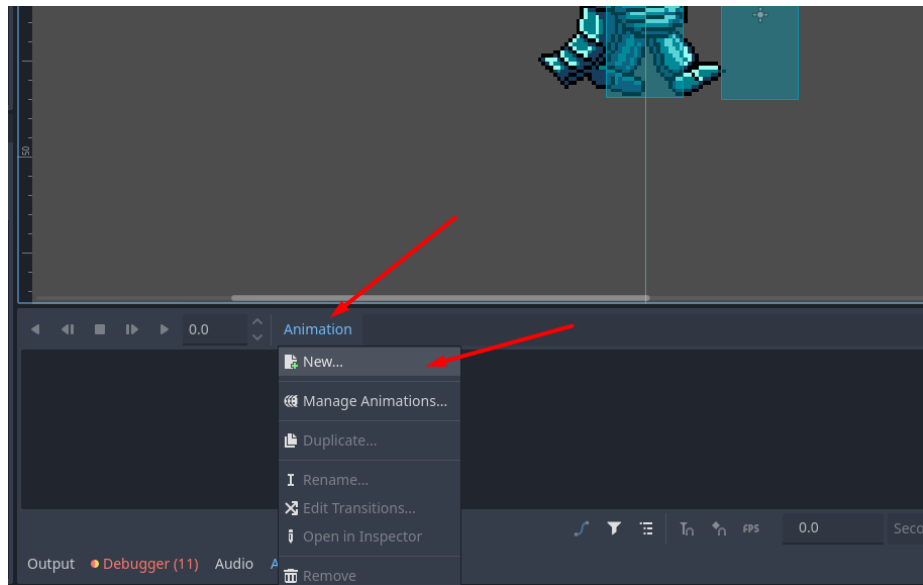
Now let's test the game.



It works! Our new character is flying! But there are new errors that say that animation “Idle” is not found. “Idle” is the equivalent of “Walk” but for flying characters, so let’s create one. Open your skin scene and select the `AnimationPlayer` node.



Create a new animation:



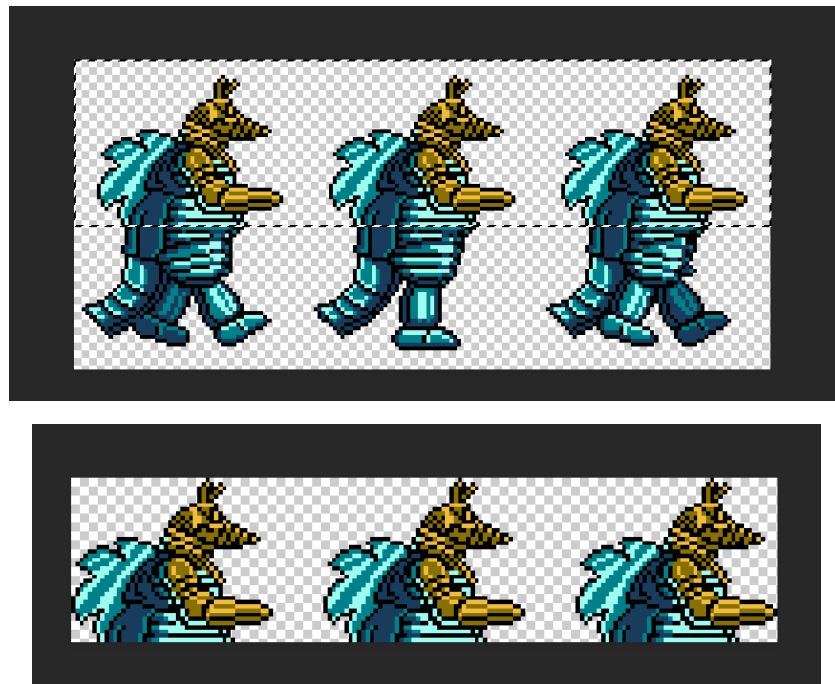
Now let's try the game again. No errors appear! Also note that this time the animation actually works like how it is described in the animation player, not by code. This is because for walking characters the walking frame speed changes depending on the direction of their movement, but since Mothra's wings don't depend on the movement, flying characters use a different behavior.

Separating the top half of the character sprites (optional)

First, let's take a look at our sprites again.



The easiest way to make sprites for the top part is to just copy half of the image and paste it into a new file.



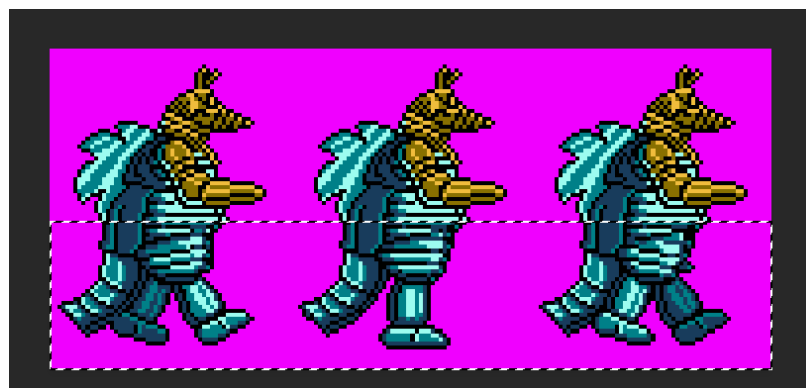
Hm, the new image looks smaller. See, when you copy an image in Photoshop, it ignores the transparent pixels. So you can change the transparent color in your image to something else, like purple, using bucket fill tool.



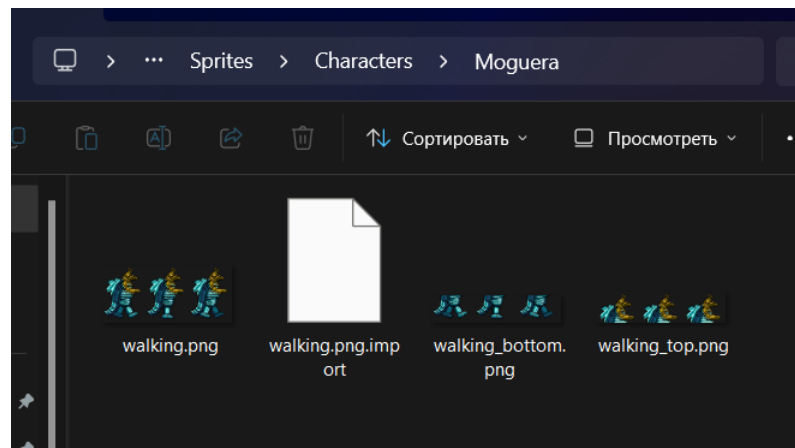
Now we can remove the background color.



Now we can do the same for the bottom part of the sprites.



Now save both parts into separate files in the character's sprites folder.



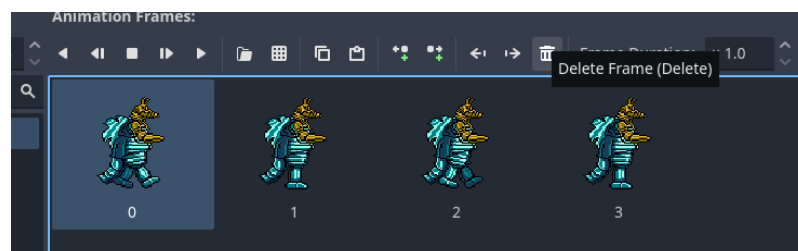
Ignore the .png.import file, it's a special file Godot generated automatically.

Since Moguera's top half stays the same throughout the animation, I changed the sprites a bit to later indicate what frame is currently shown on screen.

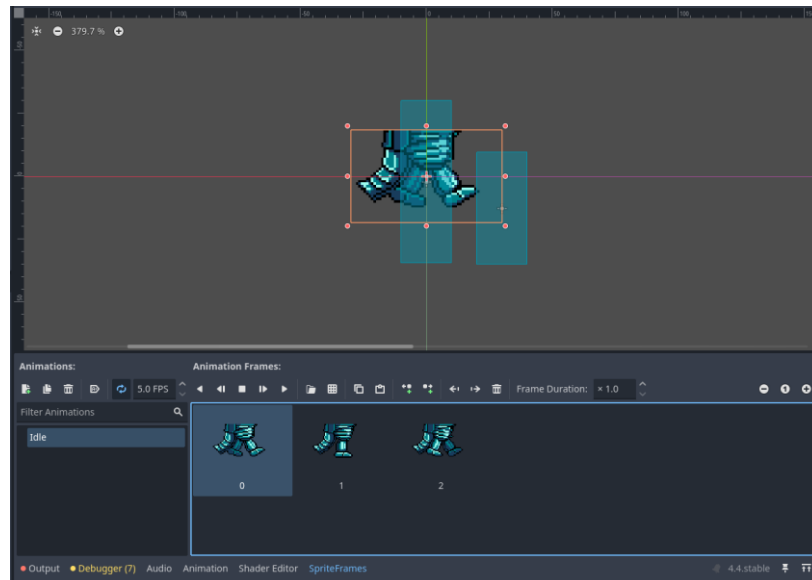


Don't judge, I'm not an artist, I'm just a programmer :D

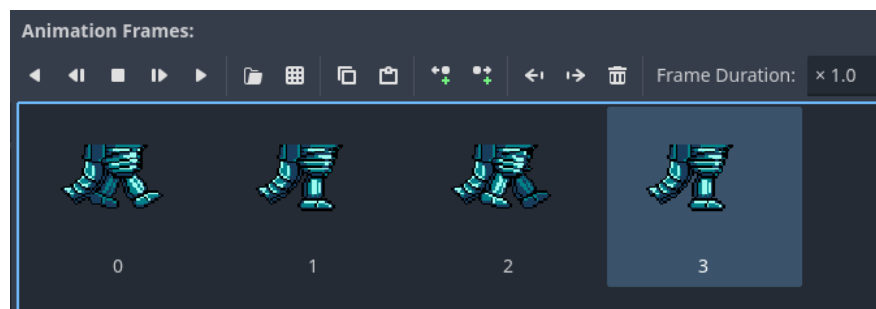
Remember how we imported the sprites in "Importing sprites into Godot" section? We can now do the same, but this time we will load the bottom half sprites into the "Body" node. But first, you will need to delete the previous frames:



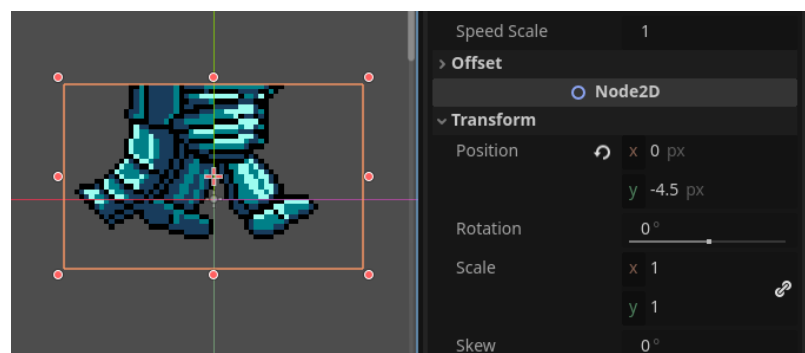
Now let's import the sprites as described in the "Importing sprites into Godot" section.



Remember how we duplicated the “standing” sprite to make the animation look alright when walking? You can do the same thing here again.

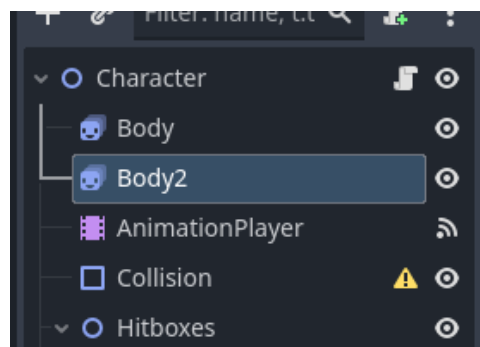
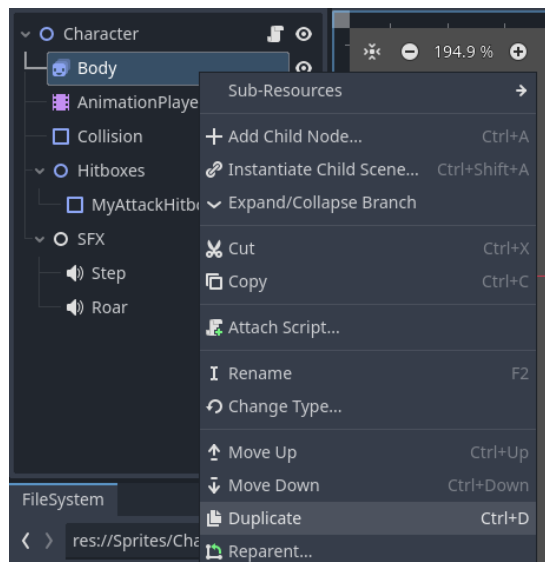


Nice! But in case if the sprites look jittery again if you zoom in/out or move the editor view, the width or height of each sprite became odd if they were even before (or they became even if they were odd before). In that case you can move the sprite's X or Y position by half a pixel and see if the shakiness stops.

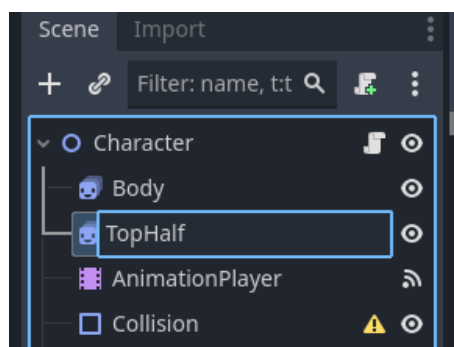
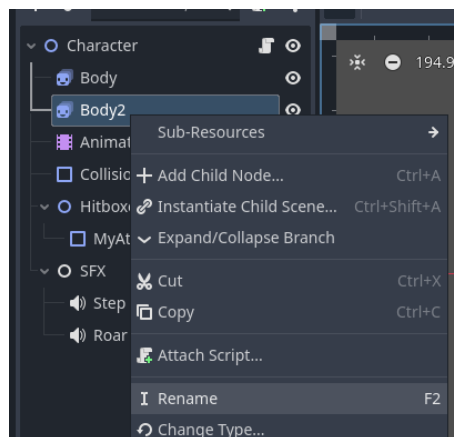


Now let's create a node for the top half. We can create a duplicate of the bottom half node to make the process simpler for this tutorial.

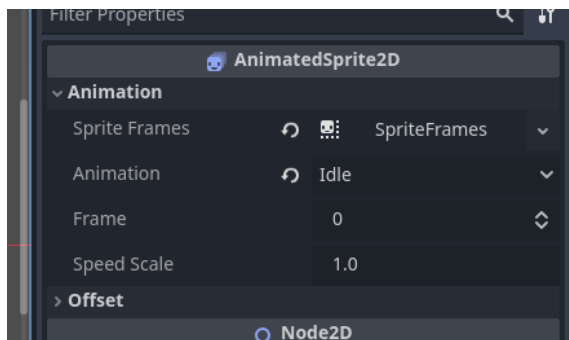
Right click on the “Body” node and select “Duplicate”:



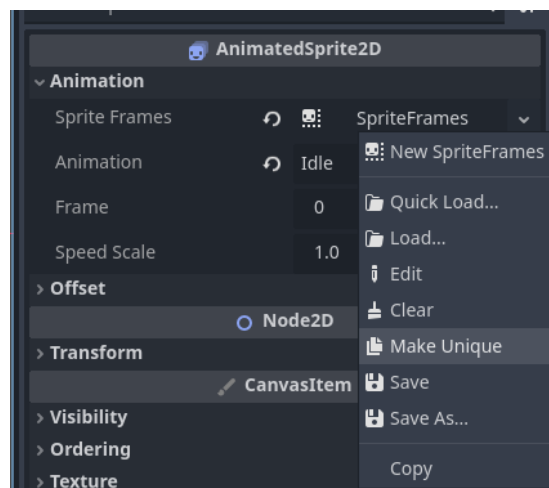
Rename it by right clicking and selecting “Rename”:



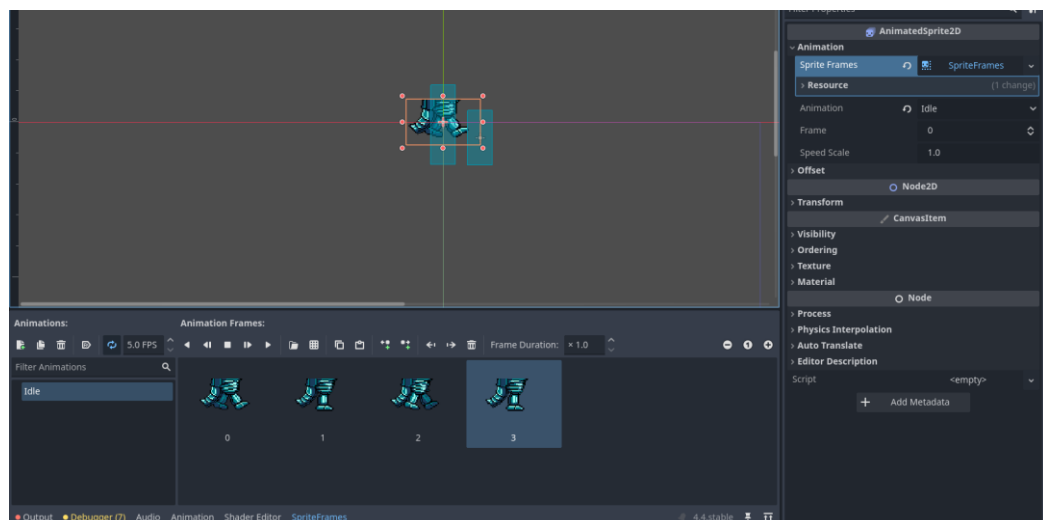
Then press Enter on your keyboard. Nice, now let's modify it. But before we do that, we need to do one more thing. Select the top half node and look at its properties:



There's a "Sprite Frames" property with a special resource type, "SpriteFrames". Since we duplicated the node, both top half and bottom half share that resource, so they use the same sprites at the moment. How to change that? We need to make the top half's sprite frames unique. Right click on "SpriteFrames" text and select "Make unique".

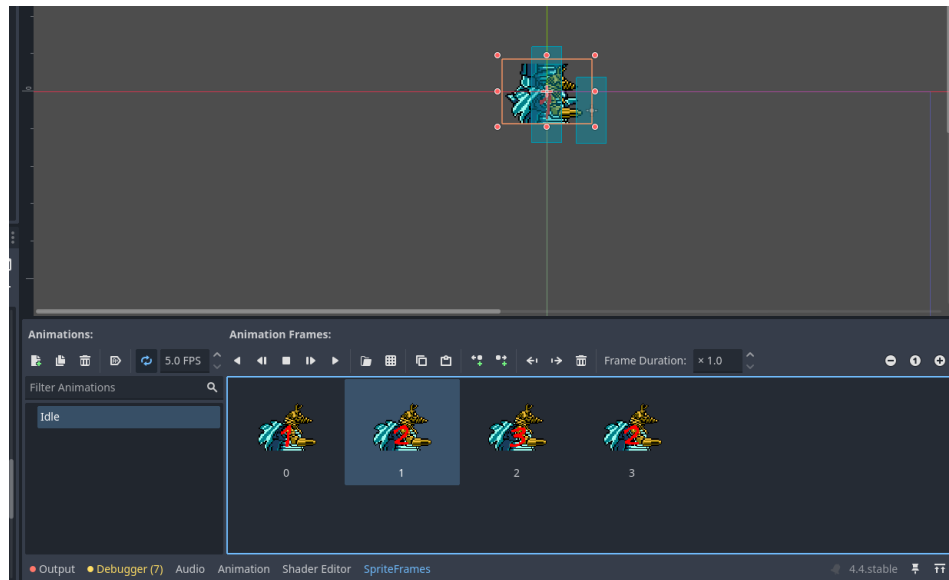


It may look like nothing happened, and after selecting the "SpriteFrames" text again we can see the same sprites of the node.



But now that resource is unique, so we can now change the sprites here without consequences. You can try importing the sprites without making that resource unique and you will see why we did that.

To import the top half sprites, you can use the same technique we used for bottom half.



Nice, they're imported. Now let's move that node until it looks alright. And if it looks jittery again, do the same thing as we did above: add half a pixel for the node's position.



Great, but the character is not in the middle of the scene, so now let's move both nodes again until it looks alright.

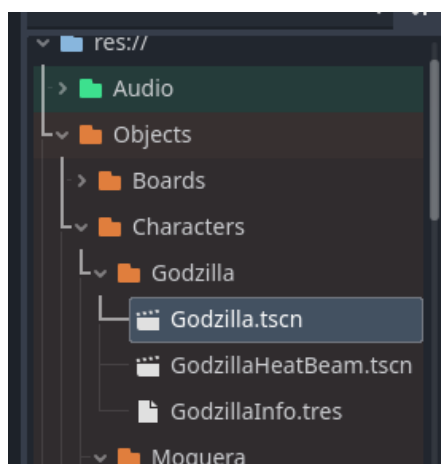


That should do it for now, let's test it in-game! If the character's sprite is a bit in the ground again, you can always move it in the skin scene.

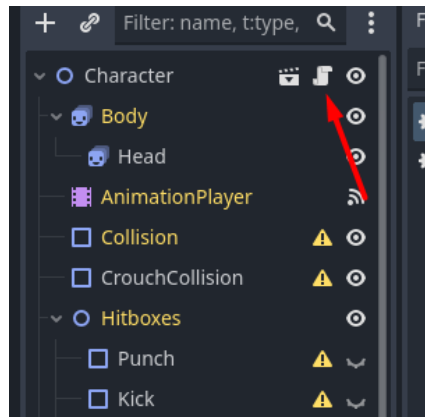


We now have bottom half and top half separate! But how would we make the top half change its frame when the character is moving?

We can look at how Godzilla skin's script does it. Open Godzilla's skin:



Click on the script:



Here it is:

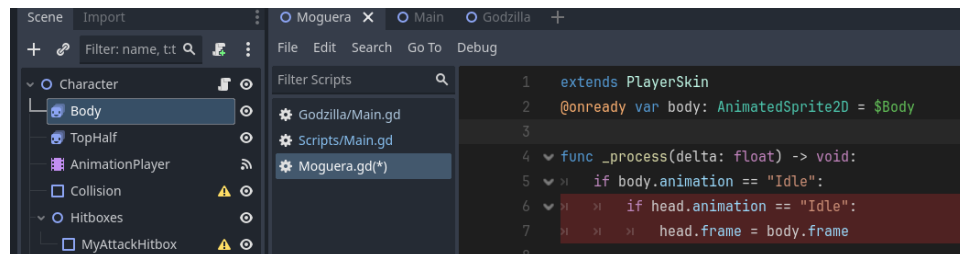
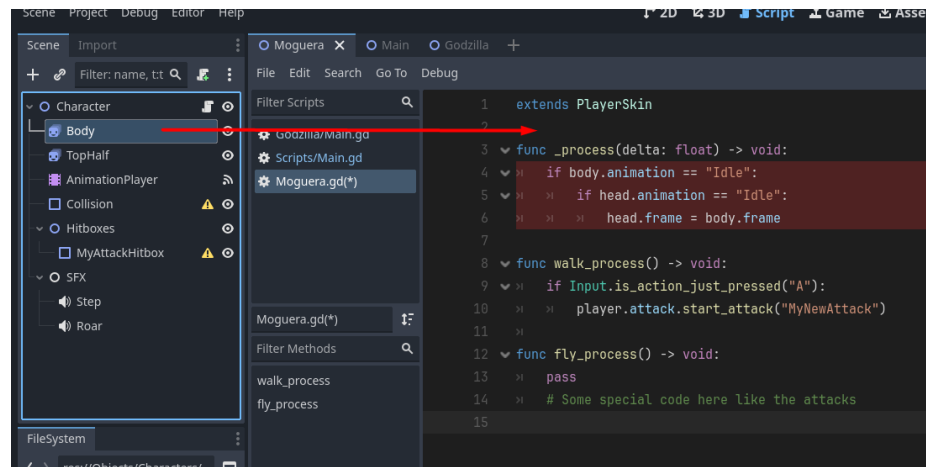
```
14 func _process(delta: float) -> void:
15     if body.animation == "Idle":
16         if body.frame == 2 or body.frame == 6:
17             head.position.y = offset_y + 1
18         else:
19             head.position.y = offset_y
20
21     if head.animation == "Idle":
22         head.frame = body.frame
```

This part of code controls the top part's frame id to be the same as the bottom part when possible (it doesn't do that if the top part's animation isn't set to walking).

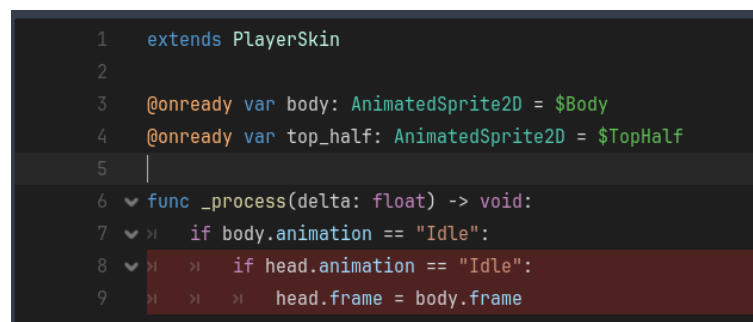
We can see that this script sets the top part's (which is called "head" here) frame property to be the same as the bottom part's ("body") frame property every frame (because it's in a special "_process" function) under the conditions that both parts are in idle state. (There's also other code between those events but that only changes the top part's Y position when Godzilla is walking, it doesn't really apply to our character here). Let's borrow the framed parts of the code to use for our character:

```
1 extends PlayerSkin
2
3 func _process(delta: float) -> void:
4     if body.animation == "Idle":
5         if head.animation == "Idle":
6             head.frame = body.frame
7
8 func walk_process() -> void:
9     if Input.is_action_just_pressed("A"):
10         player.attack.start_attack("MyNewAttack")
11
12 func fly_process() -> void:
13     pass
14     # Some special code here like the attacks
15
```

We also need to specify what “body” and “head” means. You can just drag and drop the nodes from your character skin into the code while holding Ctrl and Godot will setup the variables for you:



Do the same with the top half as well:

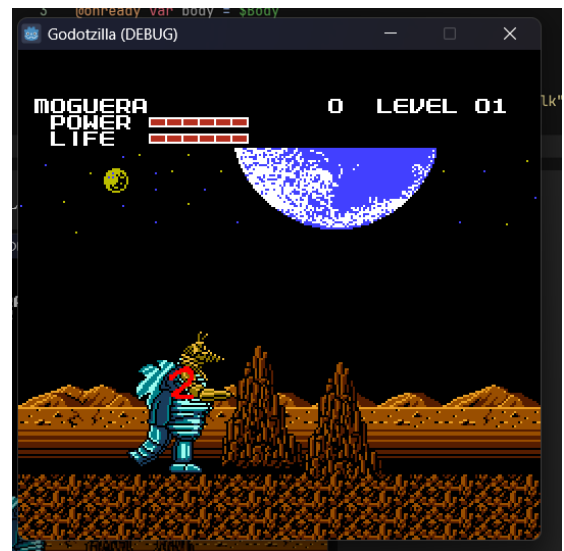
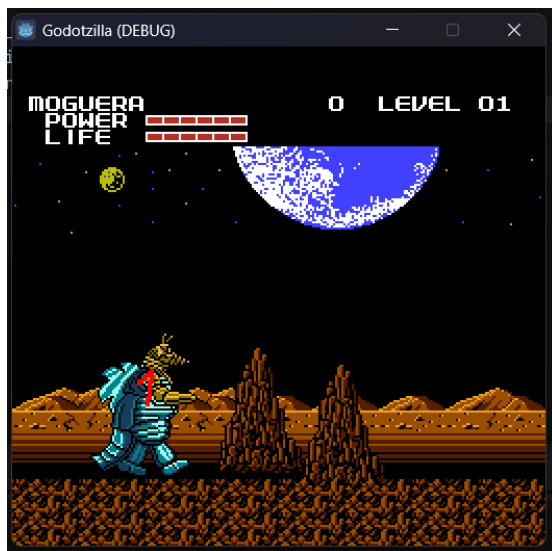


And rename it to “head”:



Now the errors are gone.

Perfect! Let's try it out in-game.



It works! Now our character has separate top and bottom parts.